7 Backoffice and multi-node integration

ACTIVE | TOTAL: orig ~7d · forecast ~8.5d | PROGRESS: spent ~0d · rem ~8.5d | forecast end 21 Dec 2026

GOAL

Run a representative multi-node SI-01 setup that consumes reference/start-time data, produces derived timing, and exchanges node-scoped timing data with a reproducible backoffice test environment while preserving local operation during outages.

RESULT

- Multiple TimingNodes retain independent state, sequence and storage.
- Node-scoped reference/start-time data can be received and applied explicitly.
- TeamID interpretation and StageTiming use defined reference-data ownership.
- Local registration continues during upstream outage.
- Pending outbound timing work resumes synchronization after recovery.
- Engineering-API and RabbitMQ transports exercise the same semantic upstream contract.

END DEMO

- Start a multi-node SI-01 test composition.
- Load synthetic RaceData and StageStartTimes for the addressed nodes.
- Feed simulated registrations and inspect TeamID/StageTiming results per node.
- Use the SI-02 Backend pane over IF-03 to send RaceData/StageStartTimes and inspect typed outbound backoffice events.
- Force acknowledgement delay/disconnect, continue local registrations, reconnect and synchronize.
- Switch the SI-02 Backend Simulator to RabbitMQ transport and repeat the recovery flow through the hosted broker against SI-01.

DOCUMENTATION

30-UC R-90 | 41-01-SSD R-85 | 32-06-ISD W-55 | 33-06-IDD W-45 | 43-01-SDD-03 W-70 | 60-SVP R-85

DOCUMENTATION / DECISIONS

D01 DONE

Promote IF-06 UpstreamData semantic contract and public JSON reference representation

IF-06 now defines transport-in...

D02 after D01 DONE

Define reference-data, TeamID, StagePassageList and StageRanking ownership/semantics

EventData owns stable profile...

D03 after D01 DONE

Define source-head, acknowledgement, REQ and LogBook synchronization semantics

Synchronization keeps two...

D04 after D01 D03 DONE

Align backoffice transport SDD plus Engineering API connector / hosted RabbitMQ verification topology

EngineeringApiUpstreamConnecto...

APPLICATION / PRODUCT

A01 DONE

Adopt and qualify the existing multi-node composition baseline

Adopt the existing multi-node...

A02 after D01 D02 V01 DONE

Implement node-scoped reference-data ingress and TeamID interpretation

Implement the Central-to-node...

A03 after A02 D02 DONE

Implement StagePassageList and StageRanking using the promoted reference-data model

Implement StageTiming using...

A04 after D01 D03 V01 NEXT

Implement transport-independent TIMINGDATA SYNC/REQ LogBook synchronization

Implement transport-independen...

A05 after D04 A02 A04 PLANNED

Implement EngineeringApiBackofficeConnector plus SI-02 Backend pane

Add EngineeringApiUpstreamConn...

A06 after D04 V02 PLANNED

Implement/qualify SI-01 RabbitMQ connector plus SI-02 RabbitMQ backend-peer transport

Implement/qualify RabbitMQ on...

VERIFICATION / TEST

V01 after A01 — DONE

Requalify multi-node addressing, isolation, persistence and recovery

Requalified on Java main...

V02 after A03 A05 — PLANNED

Verify reference-data, derived timing and synchronization through the Engineering API backoffice connector

Use the SI-02 Backend pane...

V03 after A06 — PLANNED

Verify SI-02 ↔ hosted RabbitMQ ↔ SI-01 routing/recovery end-to-end

Run ST-3 end-to-end as SI-02...

PLANNING CHANGES

- 2026-10-13 — Complete A03 on Java main revision 006a92e65e07412cb7caf603f853c90c58b86f5e. StageTiming now derives deterministic elapsed results from committed TimingData plus current EventData/ReserveTagMap/StageStartTimes state, with explicit unresolved outcomes and an explicit ZoneId for civil-time rollover. REV/lifecycle records do not become new stage passages. Ranking remains a future query policy because no ranking/tie/grouping rule is currently specified.
- 2026-10-13 — Complete A02 on Java main revision 99d1d5839c7f837d19095f3b253708728952cdc1. The UpstreamData base, ordered per-node RACE_DATA sequence, ReserveTagMap and StageStartTimes SET/DEL state, apply-before-ACK semantics, gap-to-REQ behavior, node UpstreamMessagePort and application UpstreamMessageRouter are implemented and qualified by Java verification and runtime characterization.
- 2026-10-13 — Close A01/V01 by requalifying the existing multi-topology baseline on Java main revision 7d87344725d9deb8c91bfda524b35c085b53b401. VC-ST1-007 and VC-ST1-008 both pass in the integrated -Psystem-test run, confirming independent node addressing, sequence isolation, separate LogBooks and restart recovery.

PLANNING CHANGES (CONT.)

- 2026-10-13 — Complete D04 engineering upstream transport design. EngineeringApiUpstreamConnector reuses IF-03 POST /api/v1/dev/upstream/message and the existing SI-02 event WebSocket with role=upstream; outbound semantic messages are UPSTREAM_DATA events. No second listener port or SI-02 WebSocket lifecycle is added. SI-02 acts as the Central System Simulator and the connector enters the same UpstreamData boundary as RabbitMQ.
- 2026-10-13 — Complete the D02/D03 ordered-stream model before A02 implementation. TIMING_DATA keeps the existing per-node LogBook/IF-05 seqNr. Central-to-node RACE_DATA gets its own per-node source seq and carries incremental reserveTagMap/StageStartTimes changes. SI-01 ACKs RaceData only after ordered semantic application; gaps cause bidirectional REQ from the first missing sequence. SYNC advertises a stream's source head. REQ TIMING_DATA fromSeq=1 is the semantic equivalent of the legacy receive-all/package-id 0000 request.

PLANNING CHANGES (CONT.)

- 2026-10-13 — Complete the D01 IF-06 baseline. Use one bidirectional UpstreamData model with a public/reference JSON representation: TIMING_DATA embeds structured IF-05 records, while PING uses op=REQ/ANS with optional mode=HB and optional pingId. SI-01 sends heartbeat REQ immediately on connect/reconnect and every 30 seconds; manual PING omits mode. No unanswered-PING timeout policy is fixed yet.
- 2026-10-13 — Use one SI-02 Central System Simulator with two transports: IF-03 for deterministic ST-2 and RabbitMQ for hosted ST-3. Both exercise the same semantic upstream contract; deployment-specific protocol mappings remain below the UpstreamData boundary.
- 2026-10-13 — Activate Step 7 after Step 6 closeout and adopt the existing multi-node, node-addressing, separate-LogBook and restart-recovery implementation as the baseline instead of rebuilding that topology. Continue with reference-data, synchronization and backoffice integration work from that baseline.